

Báo cáo bài tập E11.2:

convolutional_network_raw

```
In [1]: import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
import numpy as np
import matplotlib.pyplot as plt
import random
```

Some configs

```
In [2]: # Số classes trong tập MNIST
num_classes = 10

# Số epoch
epochs = 10

# Các tham số cần thiết cho quá trình training.
learning_rate = 0.01
batch_size = 64
display_step = 100

# Tham số mạng CNN
out_channel_1 = 32 # số channel của đầu ra conv thứ 1
out_channel_2 = 64 # số channel của đầu ra conv thứ 2

# Path Lưu best model
checkpoint = 'best_model_mnist.pth' # có thể để dạng *.pth
```

Dataloader

```
In [3]: # Transform image
transform=transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Load dataset từ torchvision.datasets
train_dataset = datasets.MNIST('../data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST('../data', train=False, transform=transform)
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size)
```

Model

- Input shape sẽ là: [-1, 28, 28, 1]. Ở đây -1 sẽ thể hiện batchsize, một batch thì gồm nhiều ảnh 28x28x1 (grayscale, số channel là 1 !)
- Chúng ta sẽ định nghĩa một model đơn giản gồm 2 lớp Conv đều có filter size là 3x3 và stride hãy set là 1.
- Ngoài ra sẽ có một lớp maxpool, set filter size 2x2
- Flow như sau: conv2d_1 -> relu -> conv2d_2 -> relu -> maxpool2d -> dropout -> flatten -> linear1 -> relu -> dropout -> linear2

In [4]: *# Định nghĩa model*

```
model = nn.Sequential (
    nn.Conv2d(1, 32, kernel_size=3, stride=1),
    nn.ReLU(),
    nn.Conv2d(32, 64, kernel_size=3, stride=1),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Dropout(0.25),
    nn.Flatten(),
    nn.Linear(9216, 128),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(128, 10)
)

# Load lại pretrained model (nếu có)
try:
    model.load_state_dict(torch.load(checkpoint))
except:
    print("!!! Hãy train để có checkpoint file")
```

```
In [5]: criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
best_val_loss = 999

for epoch in range(1,2):
    # Quá trình training
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % display_step == 0:
            print('Train Epoch: {} [{}/{} ({:.0f}%)]\tTrain Loss: {:.6f}'.format(
                epoch, batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item()))
    # Quá trình testing
    model.eval()
    test_loss = 0
    correct = 0
    # set no grad cho quá trình testing
    with torch.no_grad():
        for data, target in test_loader:
            output = model(data)
            output = F.log_softmax(output, dim=1) # Log softmax using F
            test_loss += F.nll_loss(output, target, reduction='sum').item()
            pred = output.argmax(dim=1, keepdim=True) # argmax để lấy predicted label, chú ý keepdim = True
            correct += pred.eq(target.view_as(pred)).sum().item()
    test_loss /= len(test_loader.dataset)
    if test_loss < best_val_loss:
        best_val_loss = test_loss
        torch.save(model.state_dict(), checkpoint) # Lưu lại model
    print("***** TEST_ACC = {}% *****".format(correct))
```

```
Train Epoch: 1 [0/60000 (0%)] Train Loss: 2.302503
Train Epoch: 1 [6400/60000 (11%)] Train Loss: 0.231514
Train Epoch: 1 [12800/60000 (21%)] Train Loss: 0.263810
Train Epoch: 1 [19200/60000 (32%)] Train Loss: 0.218469
Train Epoch: 1 [25600/60000 (43%)] Train Loss: 0.069374
Train Epoch: 1 [32000/60000 (53%)] Train Loss: 0.334526
Train Epoch: 1 [38400/60000 (64%)] Train Loss: 0.076809
Train Epoch: 1 [44800/60000 (75%)] Train Loss: 0.151130
Train Epoch: 1 [51200/60000 (85%)] Train Loss: 0.384928
Train Epoch: 1 [57600/60000 (96%)] Train Loss: 0.133841
***** TEST_ACC = 9728% *****
```

Visualize Image

```
In [6]: # Load lại model đã train
model.load_state_dict(torch.load(checkpoint))
# set eval phase
model.eval()
```

```
Out[6]: Sequential(
  (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1))
  (1): ReLU()
  (2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
  (3): ReLU()
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (5): Dropout(p=0.25, inplace=False)
  (6): Flatten(start_dim=1, end_dim=-1)
  (7): Linear(in_features=9216, out_features=128, bias=True)
  (8): ReLU()
  (9): Dropout(p=0.5, inplace=False)
  (10): Linear(in_features=128, out_features=10, bias=True)
)
```

```
In [7]: item = iter(test_loader)
```

```
In [8]: data, target = item.__next__() # Lấy một batch ra
```

```
In [9]: test_idx = random.choice(range(len(data))) # lấy index của một phần tử của một batch
```

```
In [10]: data = data[test_idx]
target = target[test_idx]
assert data.shape == (1,28,28)
```

```
In [11]: # thử predict

def plot(data, model):
    data = data.unsqueeze(0) # unsqueeze data
    output = model(data)
    output = F.log_softmax(output, dim=1) # Log softmax, chú ý dim
    pred = output.argmax(dim=1, keepdim=True) # argmax, chú ý keepdim
    print("Predict Number : ", pred[0][0].numpy())
    plt.imshow(data[0][0], cmap='gray')
    plt.show()
```

```
In [12]: plot(data, model)
```

Predict Number : 6

